3. report[0] 的修饰位图逐个和 previous[0] 比较，变化的位用 hid_to_key[224+i]（0xe0 ~ 0xe7）上报按下或松开；
4. 收尾调用 input_sync()，产生一个 EV_SYN/SYN_REPORT，一份报告就是一个完整帧；
5. 普通字符、Backspace、Enter 以及方向键的 ESC [A/B/C/D 转义序列全部送进 consoleintr()，与 UART 收到的字节走同一条路径；
6. 最后 usb_submit_urb() 重新 arm ch6。

字符映射是两张稀疏的表，一张出字符、一张出 Linux 键码：

```
// kernel/usbkbd.c，节选。两张表都按 HID usage 编号索引
static const uchar keymap[58]   = { [4]='a', [16]='m', [30]='1', [40]='\n',
                                    [42]='\b', [43]='\t', [56]='/' };
static const uchar shiftmap[58] = { [4]='A', [16]='M', [30]='!', [45]='_',
                                    [47]='{' };
static const ushort hid_to_key[256] = { [4]=KEY_A, [40]=KEY_ENTER,
                                        [82]=KEY_UP, [225]=KEY_LEFTSHIFT };
```

keymap[]/shiftmap[] 只覆盖能直接打成 ASCII 的 usage 4 ~ 56（字母、数字、常用符号、Enter/Tab/Backspace/Esc/空格）：Shift 决定取哪一张，字母还要再异或一下软件维护的 Caps Lock 状态（usbkbd_key() 里 usage 57 只翻转这个标志，自己不产生字符）；Ctrl 加字母折成 c - 'a' + 1 这样的控制字符，所以 Ctrl-C 就是 0x03。hid_to_key[] 则是 HID usage 到 Linux KEY_* 编号的映射，覆盖普通键、四个方向键和 usage 0xe0 ~ 0xe7 的修饰键；probe() 遍历它一遍，把这台设备支持的键登记成 input 设备的能力位图，于是 evdev 的用户拿到的是与 Linux 一致的编号，而不是 HID 自己的一套。两张表都没登记的 usage——F1 ~ F12、多媒体键——既不产生字符也不上报事件，直接丢掉。

evdev 队列：每个读者一份。 /dev/input/eventN 的读语义按 Linux evdev 实现，一共四点：

- **minor 稳定绑定设备。**input_register_device() 从设备表里取一个空槽，槽号就是 eventN；open() 用 inode 的 minor 反查这张表，因此不会出现所有设备都“晋升”成 event0。devtmpfs 预先创建 /dev/input/event0..event3（主设备号 INPUT = 4，minor 0 ~ 3），空槽的节点还在但 open() 失败——这正是 Linux 的 ENODEV 语义，好处是不动态增删节点时也不会把旧节点错指给后来注册的设备。
- **每个 open 一个客户端。**evdev_open() 分配一个 evdev_client（EVDEV_BUFFER = 256 条事件的环形缓冲）挂到 dev->clients，两个读者各读各的，不会互相偷事件。
- **只暴露完整帧。**packet_head 标记最后一个完整帧的末尾，read() 最多读到那里；溢出时把尾巴丢掉并插入 SYN_DROPPED，读取者据此丢弃自己缓存的按键状态，等下一个 SYN_REPORT 再恢复。
- **断开是有顺序的。**input_unregister_device() 先把设备从表里摘掉、唤醒阻塞的读者，再让旧 fd 读完已经提交的完整帧，之后 read() 才返回错误。

每条事件都是一个 struct input_event：uint16 type、uint16 code、int value，read() 一次可以拿走多条。键盘发布 EV_KEY（value 1 按下、0 松开），Boot Mouse 还会发布 BTN_LEFT/BTN_RIGHT/BTN_MIDDLE 和 EV_REL 的 REL_X/REL_Y/REL_WHEEL，末尾统一跟一个 EV_SYN/SYN_REPORT。在真机上可以直接验证：

```
$ ls /dev/input
event0  event1  event2  event3
$ cat /proc/devices        # Character devices: 4 input/event0
$ evtest /dev/input/event0 # 每次按下/松开各一条 EV_KEY，帧尾一条 EV_SYN
```

引用计数与生命周期。 input_dev 由 kalloc() 分配，初始引用属于类驱动；每个打开的 event fd 再加一个引用。usb_free_urb() 和 kfree(kbd) 发生在 remove 路径上，而 input_dev 的内存要等最后一个 fd 关闭才释放——这样“驱动已经卸载”和“用户还在读旧事件”两件事可以同时成立。注销路径因此不会睡眠等待用户空间关闭